

TRANSFORMERS

Detailed Training Notes — From Tokens and Positional Encoding to Attention, Encoders, Decoders, BERT and GPT

Purpose: These notes reorganize the training discussion into a clean learning sequence. They also correct a few technical inaccuracies so the material can be safely used for future study, projects, and interviews.

Learning Roadmap

- Why Transformers were introduced
- Tokenization, embeddings, and positional encoding
- Encoder architecture and self-attention
- Query, Key, and Value (Q, K, V)
- Scaled dot-product attention and multi-head attention
- Feed-forward networks, residual connections, and layer normalization
- Decoder architecture and masked self-attention
- Encoder–decoder cross-attention
- Encoder-only, decoder-only, and encoder–decoder model families
- BERT and GPT
- Important technical corrections and interview-ready explanations

1. Why Were Transformers Introduced?

Before Transformers, sequence models such as RNNs, LSTMs, and GRUs processed text largely in sequence. For a sentence with many tokens, this creates a long computational path and makes parallel processing difficult. LSTMs and GRUs improved memory compared with basic RNNs, but long sequences could still be expensive to process.

The Transformer architecture changed the approach. Instead of relying primarily on recurrence, it uses **attention** to determine which words are important to one another. This allows much more parallel computation during training.

Core idea: A Transformer asks, “For the word I am processing now, which other words in the sequence should I pay attention to, and by how much?”

Example

Sentence: “**The animal did not cross the street because it was too tired.**”

To understand the word “**it**”, the model should connect it strongly with “**animal**”. Self-attention helps the model calculate such relationships.

2. High-Level Transformer Architecture

The original Transformer introduced in *Attention Is All You Need* uses two major sides:

- **Encoder:** builds a contextual representation of the input sequence.
- **Decoder:** generates an output sequence, one token at a time, while using appropriate context.

For a translation task such as English → French, the encoder processes the English input and the decoder generates the French output.

Important: Not every modern Transformer uses both sides. BERT is encoder-only; GPT is decoder-only; models such as the original Transformer and T5 use encoder–decoder architectures.

3. Step 1 — Tokenization

A model does not directly process raw sentences as human-readable strings. The text is first broken into **tokens**. A token may be a whole word, a subword, or occasionally a character, depending on the tokenizer.

Example:

Text: "Transformers are powerful"
Possible tokens:
["Transform", "ers", " are", " powerful"]

Each token is then mapped to a token ID:
[15432, 388, 527, 8147]

Subword tokenization is useful because the model does not need a separate vocabulary entry for every possible complete word.

4. Step 2 — Input Embeddings

Token IDs are integer labels, but the model needs meaningful numerical representations. An **embedding layer** maps each token ID to a dense vector.

A token embedding is not simply a 0 or 1. It is a vector containing many learned numerical values. The vector dimension is determined by the model architecture.

Example conceptually:

Token: "cat"
Embedding: [0.21, -0.48, 0.77, ..., 0.13]

Token: "dog"
Embedding: [0.18, -0.41, 0.72, ..., 0.09]

Semantically related tokens may develop representations that are useful in similar contexts.

5. Why Positional Encoding Is Needed

Self-attention can process all tokens in parallel. However, attention by itself does not automatically know the original order of tokens. Word order matters because changing the order can change the meaning.

Compare: **"Dog bites man"** and **"Man bites dog."** The words are similar, but the meaning is different because the order is different.

Therefore, position information is added to token representations. In the original Transformer, sinusoidal positional encodings based on sine and cosine functions were used. Many later models use learned positional embeddings or other positional methods.

Simplified flow: Raw text → Tokens → Token IDs → Token embeddings + Position information → Transformer layers

6. The Encoder Block

A Transformer encoder layer typically contains the following major operations:

- Multi-head self-attention
- Residual/skip connection
- Layer normalization
- Position-wise feed-forward neural network
- Another residual connection and normalization

Multiple encoder layers can be stacked. The original Transformer paper used six encoder layers in its base architecture, but six is not a universal rule for all Transformers.

7. Self-Attention

Self-attention allows every token to examine other relevant tokens in the same sequence and create a context-aware representation.

Example: **"He bought the apples and ate them."** The representation of **"them"** should strongly use information from **"apples."**

The model does not perform this by hard-coded English grammar rules. It learns attention patterns from data.

8. Query, Key, and Value — Q, K, V

For each token representation, the attention layer creates three learned vectors:

- **Query (Q):** represents what the current token is looking for.

- **Key (K):** represents information used to determine how relevant another token is to the query.
- **Value (V):** contains the information that can be combined into the resulting contextual representation.

A useful analogy is a search system: the query expresses what you are looking for, keys are used for matching, and values contain the information retrieved after relevance is determined.

9. Scaled Dot-Product Attention

At a high level, attention performs four steps:

- Compare queries with keys to obtain relevance scores.
- Scale the scores for numerical stability.
- Apply Softmax to convert scores into attention weights.
- Use those weights to create a weighted combination of the value vectors.

The standard attention equation is:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

The output is a contextualized representation: a token's representation now contains information from other relevant tokens.

10. Multi-Head Attention

A single attention operation may focus on one type of relationship. Multi-head attention runs several attention heads in parallel. Different heads can learn different relationships—for example, grammatical dependencies, nearby context, or long-distance semantic relationships.

The head outputs are combined and projected into the model's representation space.

Do not interpret an attention head as having a guaranteed human-readable role. Heads may learn useful patterns, but a specific head is not automatically a fixed “grammar head” or “pronoun head.”

11. Feed-Forward Neural Network (FFN)

After attention, each token representation passes through a feed-forward neural network. The same FFN parameters are applied independently to each token position within that layer.

A simplified form is:

$$\text{FFN}(x) = \text{activation}(xW_1 + b_1)W_2 + b_2$$

The attention mechanism mixes information across tokens. The feed-forward network then transforms each token's representation.

12. Residual or Skip Connections

A residual connection adds a layer's input back to the transformed output. This helps information and gradients flow through deep networks.

Simplified idea: **Output = Input + Sublayer(Input)**

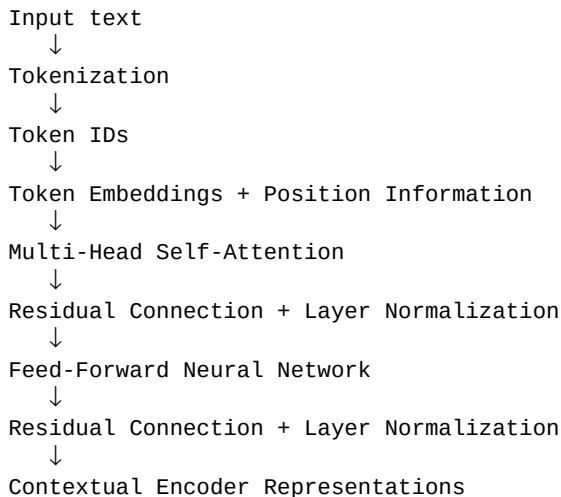
This does not mean that the model simply copies the original sentence. It means the original representation is preserved as part of the layer computation.

13. Layer Normalization

Layer normalization helps stabilize the activations inside the network and supports more reliable training.

The training transcript describes normalization mainly as converting “very big and very small numbers into normal numbers.” That is an oversimplification. More precisely, layer normalization normalizes activations across the feature dimension for each token representation, followed by learned scale and shift parameters.

14. Complete Encoder Flow



15. The Decoder

The decoder is designed to generate an output sequence. During autoregressive generation, it predicts the next token based on the tokens generated so far.

In the original encoder–decoder Transformer, a decoder layer includes:

- Masked self-attention over the target sequence
- Cross-attention to encoder outputs
- Feed-forward neural network
- Residual connections and layer normalization around the sublayers

16. Masked Self-Attention

During next-token prediction, the model must not look at future target tokens. A causal mask blocks access to future positions.

Suppose the target is: “**Joey eats pizza.**” When predicting “**eats**”, the decoder may use “**Joey**” but should not be allowed to inspect “**pizza**” as a future answer.

This prevents information leakage during training and makes the training objective consistent with autoregressive generation.

17. Cross-Attention

In an encoder–decoder Transformer, the decoder also needs information from the source sentence. Cross-attention connects the decoder to the encoder representations.

In the standard description of cross-attention:

- Queries come from the decoder's current hidden representations.
- Keys and values come from the encoder outputs.

This lets the decoder ask: “Which parts of the input sentence are relevant to the output token I am generating now?”

18. Linear Layer and Softmax

At the output, the decoder's hidden representation is projected to scores over the vocabulary. A Softmax operation can convert these scores into a probability distribution over possible next tokens.

Example:

```
Possible next token probabilities:
"pizza"    → 0.62
"food"     → 0.17
"quickly"  → 0.05
...
```

A decoding strategy then selects or samples a token. The process repeats until a stopping condition is reached.

19. English-to-French Translation — End-to-End Example

Input: “**The cat sat on the mat.**”

- The English sentence is tokenized.
- Tokens are converted to embeddings and combined with position information.
- The encoder creates contextual source representations.
- The decoder begins with a start condition or initial target context.
- Masked self-attention uses only previously available target tokens.
- Cross-attention uses the encoder representations to focus on relevant source information.
- The output layer produces next-token probabilities.
- A token is generated, and the process repeats until the translation is complete.

20. Three Major Transformer Model Families

Architecture	Main Strength	Examples
Encoder-only	Understanding and representation of input text	BERT and related models
Decoder-only	Autoregressive text/code generation	GPT family and many modern generative LLMs
Encoder–decoder	Sequence-to-sequence transformation	Original Transformer, T5, BART

21. BERT

BERT stands for **Bidirectional Encoder Representations from Transformers**. It is fundamentally an **encoder-only** Transformer model.

“Bidirectional” means BERT can build a token representation using context from both the left and right sides of that token. This is useful for understanding tasks.

Typical BERT-style uses include:

- Text classification
- Sentiment analysis
- Named entity recognition
- Extractive question answering
- Feature/embedding generation

Correction: BERT's bidirectionality should not be explained as ordinary left-to-right next-word prediction plus right-to-left next-word prediction. Its contextual representations can attend to tokens on both sides, subject to the pretraining setup.

22. GPT

GPT stands for **Generative Pre-trained Transformer**. The GPT family is fundamentally based on a **decoder-only Transformer** architecture.

GPT-style models use causal/masked self-attention so that each position predicts based on previous available tokens rather than future tokens.

Typical capabilities include:

- Text generation
- Conversation
- Code generation
- Summarization
- Rewriting
- Question answering
- Many tasks expressed through prompting and instruction following

Major correction to the training transcript: GPT-1 and GPT-2 were not encoder-only models. They were also decoder-only Transformer models. Later GPT generations did not become GPT models by simply adding an encoder to an earlier encoder-only architecture.

23. RNN vs LSTM vs Transformer

Feature	RNN	LSTM	Transformer
Sequence processing	Mostly sequential	Mostly sequential	Highly parallelizable during training
Long-range dependencies	Difficult	Improved	Attention provides direct token-to-token interactions
Core mechanism	Recurrent hidden state	Gated recurrent memory	Attention

Feature	RNN	LSTM	Transformer
Training scalability	Limited by sequential path	Limited by sequential path	Generally better parallelism

24. Simple Python Demonstration — Token Embeddings and Position Information

The following is a small educational example. It is not a complete production Transformer.

```
import torch
import torch.nn as nn

# Number of tokens available in our toy vocabulary
vocab_size = 1000

# Size of each token vector
embedding_dim = 64

# Maximum sequence length supported in this toy example
max_length = 50

# Converts token IDs into learned token vectors
token_embedding = nn.Embedding(vocab_size, embedding_dim)

# Creates learned vectors for token positions: 0, 1, 2, ...
position_embedding = nn.Embedding(max_length, embedding_dim)

# Example token IDs for one sentence
token_ids = torch.tensor([[12, 45, 89, 7]])

# Create position IDs matching the sequence length
positions = torch.arange(token_ids.size(1)).unsqueeze(0)

# Look up token meaning representations
token_vectors = token_embedding(token_ids)

# Look up position representations
position_vectors = position_embedding(positions)

# Combine token information with position information
transformer_input = token_vectors + position_vectors

print(transformer_input.shape)
# Output shape: [batch_size, sequence_length, embedding_dim]
# Here: [1, 4, 64]
```

Code explanation:

- **nn.Embedding** is a learnable lookup table.
- **token_embedding** maps each token ID to a dense vector.
- **position_embedding** maps each sequence position to a position vector.
- **token_vectors + position_vectors** gives the model both token information and position information.
- The output shape keeps the batch dimension, sequence length, and embedding dimension.

25. Simplified Attention Code

```
import torch
import math
```

```

# Toy Query, Key, and Value tensors
Q = torch.randn(1, 4, 64)
K = torch.randn(1, 4, 64)
V = torch.randn(1, 4, 64)

# Step 1: Compare Queries with Keys
scores = Q @ K.transpose(-2, -1)

# Step 2: Scale by square root of key dimension
scores = scores / math.sqrt(K.size(-1))

# Step 3: Convert scores into attention weights
attention_weights = torch.softmax(scores, dim=-1)

# Step 4: Create weighted combinations of Values
output = attention_weights @ V

print(output.shape)
# [1, 4, 64]

```

Code explanation:

- Q and K are compared using matrix multiplication.
- The scores are scaled to help keep the Softmax calculation numerically well behaved.
- Softmax converts the scores into relative attention weights.
- Those weights are applied to V to create contextualized output vectors.

26. Common Misunderstandings to Avoid

- **Transformer does not mean encoder + decoder in every model.** Transformer-based models can be encoder-only, decoder-only, or encoder–decoder.
- **GPT is not encoder-only.** GPT is a decoder-only Transformer family.
- **BERT is not primarily a generative decoder.** It is an encoder model designed for contextual understanding.
- **Feed-forward layers do not directly convert mathematical values into words.** Vocabulary prediction occurs after the model representation is projected to vocabulary logits and a decoding process chooses tokens.
- **Layer normalization is more precise than simply reducing large numbers.** It normalizes internal activations according to its defined operation.
- **Attention is not guaranteed to provide a perfect human explanation of model reasoning.** It is a learned computational mechanism.

27. Interview-Ready Answers

Q: What is a Transformer?

A Transformer is a neural-network architecture that uses attention mechanisms to model relationships between tokens. Unlike recurrent models, it can process sequence positions with much more parallelism during training and can capture long-range dependencies effectively.

Q: Why is positional encoding needed?

Self-attention alone does not inherently represent token order. Positional information is added so the model can distinguish different positions and understand sequence order.

Q: What is self-attention?

Self-attention lets each token calculate how relevant other tokens in the same sequence are and use that information to build a context-aware representation.

Q: What are Query, Key, and Value?

The Query represents what a token is looking for, the Key is used to measure relevance, and the Value contains the information combined according to the calculated attention weights.

Q: What is the difference between BERT and GPT?

BERT is primarily an encoder-only, bidirectional contextual model used for understanding tasks. GPT is a decoder-only, autoregressive model designed for generation and predicts tokens using previous available context.

Q: What is masked self-attention?

Masked or causal self-attention prevents a token from accessing future tokens during next-token prediction. This prevents information leakage and supports autoregressive generation.

28. Final Revision Summary

- Tokenization breaks text into model-processable units.
- Embeddings convert token IDs into dense vectors.
- Position information tells the model where tokens occur.
- Self-attention calculates relationships among tokens.
- Q, K, and V are learned representations used in attention.
- Multi-head attention learns multiple attention patterns in parallel.
- Feed-forward networks transform each contextualized token representation.
- Residual connections and layer normalization support stable deep networks.
- Encoder-only models are strong for understanding and representation tasks.
- Decoder-only models are strong for autoregressive generation.
- Encoder–decoder models are natural for sequence-to-sequence transformations.
- BERT is encoder-only; GPT is decoder-only.

One-line memory trick: Encoder = contextual understanding, Decoder = autoregressive generation, Attention = deciding what information matters for the current representation.